Deep Learning

25-05-26

ML - works mostly on tabular data

DL - works with tabular + non-tabular / unstructured data

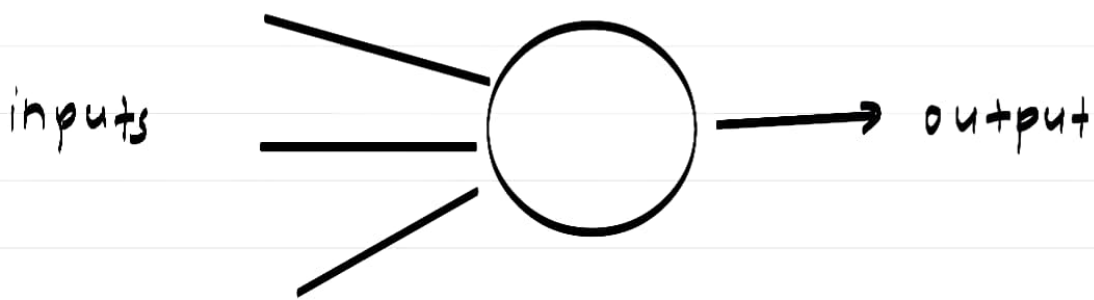
Unstructured data → text, images, videos, audios, etc

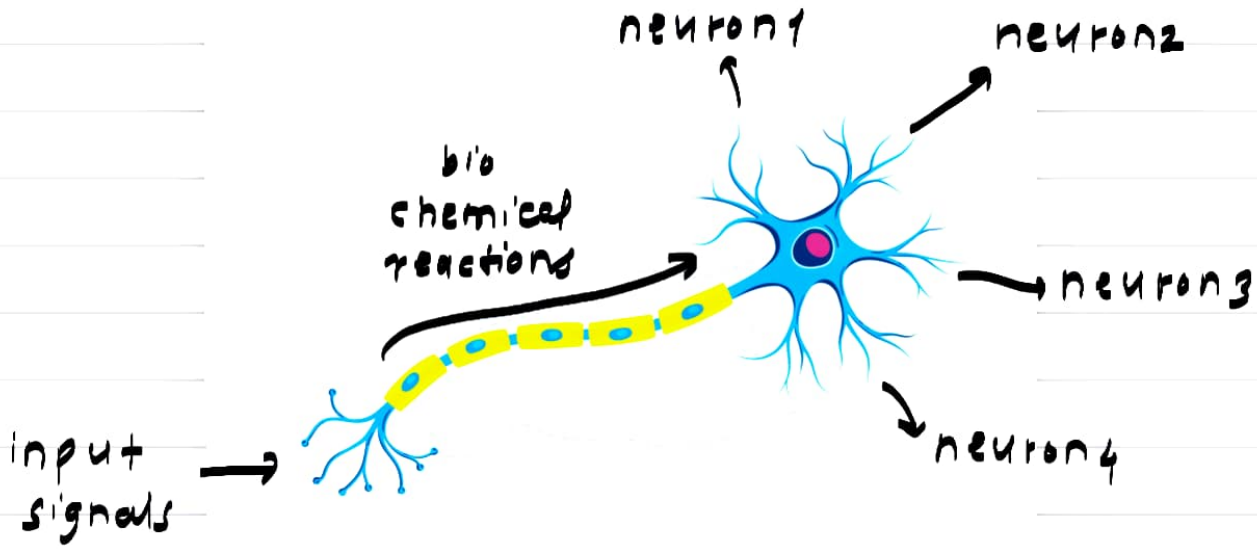
ML solved most of the easier automation tasks in AI but it was not capable enough to solve very complex automations - hence DL was developed.

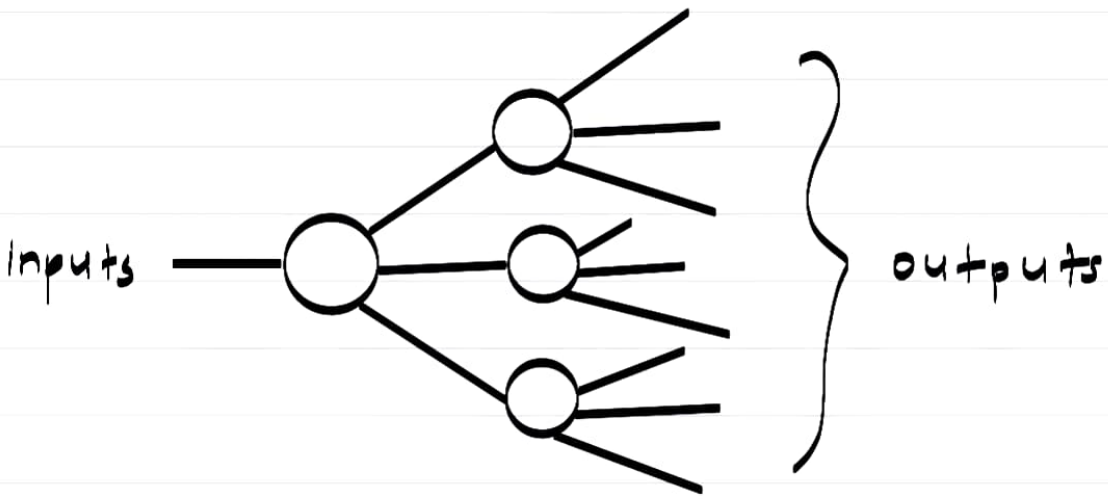
Deep Learning is based on a mathematical unit called perceptron.

Neuron cells in humans/ living organisms → intelligence

perceptron → mathematical replication of neuron cell → artificial intelligence



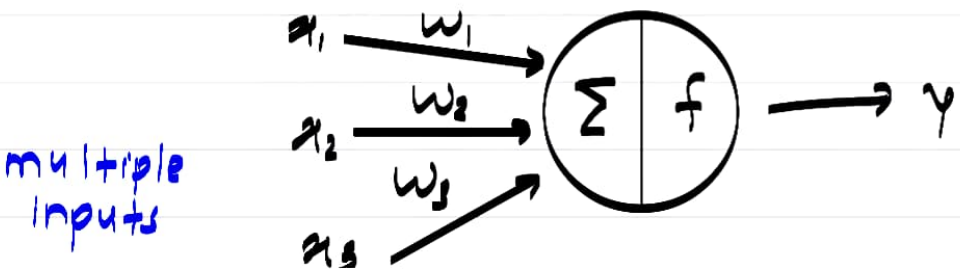




- every neuron receives input from neurons attached to it, it processes that input and generates output.
- then it passes the output to neurons next to it.

Perceptron

- proposed in 1943



output from perceptron is generated in 2 steps

1) weighted aggregation of inputs

$$Z = w_1 x_1 + w_2 x_2 + w_3 x_3 + b$$

2) Application of activation function

$$y = f(z)$$

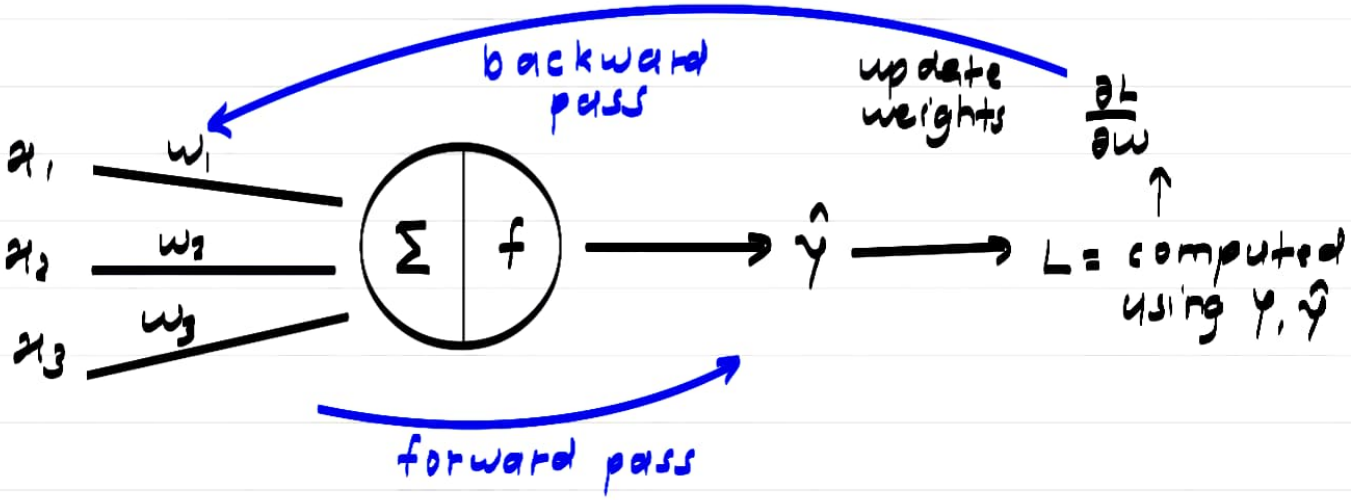
$$y = f(w_1 x_1 + w_2 x_2 + w_3 x_3 + b)$$

First we'll see how to train a single perceptron and later we'll see how to train a neural network having multiple perceptrons

perceptrons = neurons

Training of perceptrons involves 2 main steps

1) Forward propagation
2) Backward propagation



Forward Propagation

$$z = w_1 x_1 + w_2 x_2 + w_3 x_3 + b$$

$$\hat{y} = f(z)$$

Backward Propagation

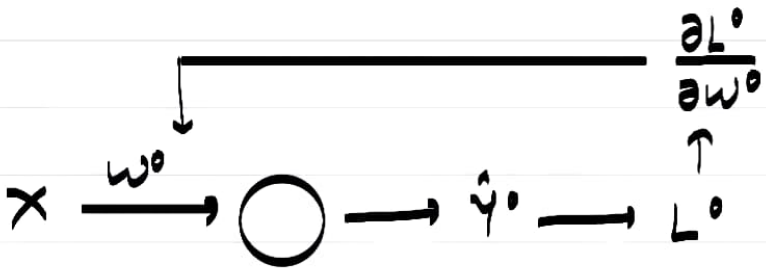
1) compute loss by comparing y_{act} and y_{pred} (y and $\hat{y}$)

2) perform Gradient Descent

3) Update model parameters to minimize loss. (w_1 w_2 w_3 b)

perceptron training ⟶ multiple iterations of forward pass and backward pass

$x \rightarrow (x_1, x_2, x_3)$



iteration 1

$$w^1 = w^0 - \alpha \frac{\partial L^0}{\partial w^0}$$



iteration 2

$$w^2 = w^1 - \alpha \, \partial L^1/\partial w^1$$

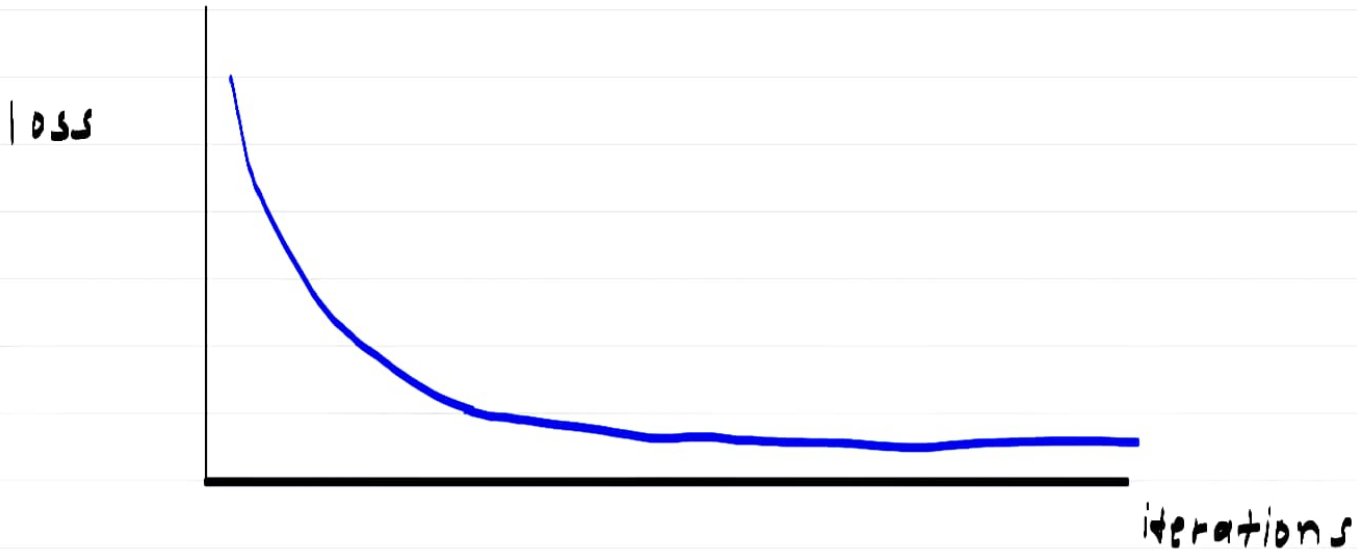
Stopping criteria:

Perceptron training will stop

1) we reach at minima → convergence

$$\frac{\partial L}{\partial w} \approx 0$$

2) Loss stops to reduce further



Regression using perceptron

Data matrix → n datapoints and m features

$$X = \begin{bmatrix} x_{11} & x_{21} & x_{31} \cdots & x_{n1} \\ x_{12} & x_{22} & x_{32} & x_{n2} \\ x_{13} & x_{23} & x_{33} & x_{n3} \\ x_{14} & x_{24} & x_{34} & x_{n4} \\ \vdots & \vdots & \vdots & \vdots \\ x_{1m} & x_{2m} & x_{3m} & x_{nm} \end{bmatrix}$$

$d_1 \quad d_2 \quad d_3 \quad d_n$

$m \times n$ matrix

we add a new feature x_0 in X to accomodate bias term

$$x_{10} = x_{20} = x_{30} \dots = x_{n0} = 1$$

$$X = \begin{bmatrix} x_{10} & x_{20} & x_{30} & \cdots & x_{n0} \\ x_{11} & x_{21} & x_{31} & & x_{n1} \\ x_{12} & x_{22} & x_{32} & & x_{n2} \\ x_{13} & x_{23} & x_{33} & & x_{n3} \\ \vdots & \vdots & \vdots & & \vdots \\ x_{1m} & x_{2m} & x_{3m} & & x_{nm} \end{bmatrix}_{(m+1)\times n}$$

$d_1 \quad d_2 \quad d_3 \quad d_n$

this data matrix has m+1 features after adding x_0

perceptron will have separate weight assigned to each of these features

$$w = [w_0 \; w_1 \; w_2 \; w_3 \; w_4 \dots w_m]_{1\times(m+1)}$$

weights as row vector = $\vec{w}$

for every datapoint in X, we need perceptron output using w.

for datapoint d_1

$$z_1 = w_0 x_{10} + w_1 x_{11} + w_2 x_{12} + w_3 x_{13} \dots + w_m x_{1m}$$

$$z_1 = \vec{w} \cdot d_1$$

$$\hat{y}_1 = f(z_1)$$

for a_2

$$z_2 = w_0 x_{20} + w_1 x_{21} + w_2 x_{22} + w_3 x_{23} - \ldots - + w_m x_{2m}$$

$$z_2 = \vec{w} \cdot a_2$$

$$\hat{y}_2 = f(z_2)$$

⋮

for a_n

$$z_n = w_0 x_{n0} + w_1 x_{n1} + w_2 x_{n2} + w_3 x_{n3} \ldots - + w_m x_{nm}$$

$$z_n = \vec{w} \cdot a_n$$

$$\hat{y}_n = f(z_n)$$

above operations are simply a single vector-matrix multiplication

$$Z = w X$$

$1 \times n$ $\quad$ $1 \times (m+1)$ $\quad$ $(m+1) \times n$

$$\hat{Y} = f(Z)$$

$1 \times n$ $\quad$ $1 \times n$

$$Z = wX = [z_1 \; z_2 \; z_3 \; z_4 \; z_5 \ldots z_n]$$

$$\hat{Y} = f(Z) = [\hat{y}_1 \; \hat{y}_2 \; \hat{y}_3 \; \hat{y}_4 \; \hat{y}_5 \ldots \hat{y}_n]$$

$$Y = [y_1 \; y_2 \; y_3 \; y_4 \; y_5 \ldots y_n]$$

} loss

$y = f(z)$

in regression, we use $f \rightarrow$ linear activation function

Linear Activation Function:

$$y = f(z) = z$$

in regression using perceptron

$$z = wx$$

$$\hat{y} = f(z) = z = wx$$

$$\boxed{\hat{y} = wx}$$

$$\hat{y} = w_0 x_0 + w_1 x_1 + w_2 x_2 \ldots - w_m x_m$$

ML Linear regression $\Longrightarrow$ regression using single perceptron

Perceptron is very simple model which can't handle complex tasks

hence we combine multiple perceptrons in layers and create powerful Artificial Neural Network ANN

When multiple neurons are combined through multiple layers to form a complex network, its called ANN - artificial neural network.